

Databricks Notebook for E-Commerce Cohort Customer Analysis 2024 - 2025 - SQL and Python/PySpark codes

```
-- Create a new schema with the name ecom_data in the workspace

CREATE SCHEMA IF NOT EXISTS workspace.ecom_data;
-- Show tables in the new schema

SHOW TABLES IN workspace.ecom_data;
/* Upload the file ecommerce...2024_rebalanced.csv in a created volume
cohorts in the created schema ecom_data. Create directly a Delta Table with
the name src_orders_2024 in the new schema ecom_data*/

CREATE OR REPLACE TABLE workspace.ecom_data.src_orders_2024
USING DELTA
AS SELECT * FROM read_files
('/Volumes/workspace/ecom_data/cohorts/ecom_orders_2024_rebalanced -
ecom_orders_2024_rebalanced.csv', format => 'csv', header => true,
inferSchema => true);

-- Count rows and confirm min/max dates are in 2024

SELECT
  COUNT (*),
  MIN (order_date) AS min_order_date,
  MAX (order_date) AS max_order_date
FROM workspace.ecom_data.src_orders_2024;

%python

# Create a Spark Dataframe from the raw data table and show this table

dfraw = spark.table("workspace.ecom_data.src_orders_2024")
dfraw.display()

%python

# Create a Bronze table called raw_orders

bronze_path = '/Volumes/workspace/ecom_data.raw_orders/'
```

```

%python

# Create the raw Delta table directly in the schema as Bronze-Layer with the
table name raw_orders

dfraw = spark.table("workspace.ecom_data.src_orders_2024")
dfraw.write.format('delta').mode('overwrite').saveAsTable('workspace.ecom_data.raw_orders')
dfraw.display()

%python

# Standardize formatting while reading Bronze-Layer from Delta table in
workspace.ecom_data schema
from pyspark.sql.functions import trim, col, cast

dfraw = spark.table('workspace.ecom_data.raw_orders')

# remove spaces in the columns customer_id and order_id and ensure
numeric/decimal format in the column sales
df_bronze = dfraw.withColumn("customer_id", trim(col("customer_id"))) \
    .withColumn("order_id", trim(col("order_id"))) \
    .withColumn("sales", col("sales").cast("decimal(18,2)"))

df_bronze.display()

%python

# Check the Bronze layer:
# total rows
# # unique orders
# # unique customers
# min/max order date
# Hint:
# Use COUNT(*), COUNT(DISTINCT ...), MIN(...), MAX(...). */

from pyspark.sql import functions as F

# Load Bronze-Layer as Spark DataFrame
df_bronze = spark.table('workspace.ecom_data.raw_orders')

# Check of the Key figures

```

```

summary = df_bronze.agg(
    F.count("*").alias("total_rows"),
    F.countDistinct("customer_id").alias("unique_customers"),
    F.countDistinct("order_id").alias("unique_orders"),
    F.min("order_date").alias("min_order_date"),
    F.max("order_date").alias("max_order_date")
)

display(summary)

%python

from pyspark.sql import functions as F
from pyspark.sql.window import Window

# Load Bronze table
df_bronze = spark.table('workspace.ecom_data.raw_orders')

# Remove invalid rows
cleaned = df_bronze.filter(
    (F.col('customer_id').isNotNull()) &
    (F.col('order_id').isNotNull()) &
    (F.col('order_date').isNotNull()) &
    (F.col('sales').isNotNull()) &
    (F.col('sales') > 0)
)

# Deduplicate orders using window function
window = Window.partitionBy('order_id').orderBy(F.col('order_date').asc())
cleaned = cleaned.withColumn('row_num', F.row_number().over(window))
deduped = cleaned.filter(F.col('row_num') == 1).drop('row_num')

# Save the deduped data as a new Delta table in the schema as Silver--Layer
with the table name clean_orders

deduped.write.format('delta').mode('overwrite').saveAsTable('workspace.ecom_data.clean_orders')
deduped.display()

```

```

%python

from pyspark.sql import functions as F

# Load Silver table

df_silver = spark.table('workspace.ecom_data.clean_orders')

# Check summary statistics
summary = df_silver.agg(
    F.count("*").alias("rows"),
    F.countDistinct("order_id").alias("unique_orders"),
    F.countDistinct("customer_id").alias("customers"),
    F.min("order_date").alias("min_order_date"),
    F.max("order_date").alias("max_order_date")
)
display(summary)

-- Final check: Use SHOW TABLES to confirm the tables exist.

SHOW TABLES in workspace.ecom_data;

-- Preview clean_orders with LIMIT 10 to confirm columns look correct

SELECT * FROM workspace.ecom_data.clean_orders LIMIT 10;

%python

# Create a Gold-Layer with the Delta Table and the name cohort_customers
directly in the schema ecom_data

# We need for this table: Each customer's first purchase date
# Each customer's second purchase date (if any): # the earliest order date
after the first purchase
# The number of days between the first and second # purchases
# The cohort month (month of first purchase)

from pyspark.sql import functions as F
from pyspark.sql.window import Window

```

```

df_silver = spark.table('workspace.ecom_data.clean_orders')

# assign a unique, sequential number to each order belonging to a specific
customer, ordered by the date the purchase was made.

w = Window.partitionBy('customer_id').orderBy('order_date')

# The function F.row_number () assigns the number 1 to the earliest order, 2
to the second, and so on.
orders = df_silver.withColumn('row_num', F.row_number().over(w))

# Since I previously ordered the window by order_date, the row with row_num
== 1 is mathematically the earliest purchase for that specific customer and
keep only the column which is necessary for this analysis, the customer_id as
primary key and renamed these first order dates to first_purchase_date

first_orders = orders.filter(F.col('row_num') == 1).select(
    'customer_id',
    F.col('order_date').alias('first_purchase_date')
)

# It does the same with the second_purchase_date, see above and take the
row_num == 2

second_orders = orders.filter(F.col('row_num') == 2).select(
    'customer_id',
    F.col('order_date').alias('second_purchase_date')
)

# Use Left Join for the two tables

cohort = first_orders.join(second_orders, 'customer_id', 'left')

# take the first purchase date as new column cohort_month and it should have
only the year and month with the date_trunc function

cohort = cohort.withColumn(
    'cohort_month',
    F.date_trunc('month', F.col('first_purchase_date'))
)

# Calculate the difference between the first and second purchase with the
datediff function

```

```

cohort = cohort.withColumn(
    'days_to_second_purchase',
    F.datediff('second_purchase_date', 'first_purchase_date')
)

cohort.write.format('delta').mode('overwrite').saveAsTable('workspace.ecom_data.cohort_customers')
cohort.display()

-- Sample rows
SELECT * FROM workspace.ecom_data.cohort_customers LIMIT 10;

-- Validate: Total customers, repeaters, one-time customers

SELECT
    COUNT(*) AS total_customers,
    COUNT(CASE WHEN second_purchase_date IS NOT NULL THEN 1 END) AS repeaters,
    COUNT(CASE WHEN second_purchase_date IS NULL THEN 1 END) AS
one_time_customers
FROM workspace.ecom_data.cohort_customers;

/* Create directly a new renamed Delta table src_orders_2025 in the schema
ecom_data from the new uploaded file ecommcerc...2025_rebalanced.csv in the
volume cohorts */

CREATE OR REPLACE TABLE workspace.ecom_data.src_orders_2025
USING DELTA
AS SELECT * FROM read_files
('/Volumes/workspace/ecom_data/cohorts/ecom_orders_incremental_2025_rebalanced - ecom_orders_incremental_2025_rebalanced.csv', format => 'csv', header =>
true, inferSchema => true)

-- Count rows and confirm min/max dates are in 2025

SELECT
    COUNT (*),
    MIN (order_date) AS min_order_date,
    MAX (order_date) AS max_order_date
FROM workspace.ecom_data.src_orders_2025;

```

```

-- Append rows from src_orders_2025 into your existing Bronze table
raw_orders

INSERT INTO workspace.ecom_data.raw_orders (
  row_id,
  customer_id,
  order_date,
  order_id,
  sales
)
SELECT
  row_id,
  customer_id,
  order_date,
  order_id,
  sales
FROM workspace.ecom_data.src_orders_2025;

%python
# Standardize formatting with data from 2025 while reading Bronze-Layer from
Delta table in workspace.ecom_data schema

from pyspark.sql.functions import trim, col, cast

dfraw = spark.table('workspace.ecom_data.raw_orders')

# remove spaces in the columns customer_id and order_id and ensure
numeric/decimal format in the column sales
df_bronze = dfraw.withColumn("customer_id", trim(col("customer_id"))) \
  .withColumn("order_id", trim(col("order_id"))) \
  .withColumn("sales", col("sales").cast("decimal(18,2)"))

df_bronze.display()

-- Count rows and confirm min/max dates are in 2024 - 2025

```

```

SELECT
    COUNT(*) AS total_rows,
    MIN(order_date) AS min_order_date,
    MAX(order_date) AS max_order_date
FROM workspace.ecom_data.raw_orders;

%python

# Check the Bronze layer with the new data from 2025 again:
# total rows
# # unique orders
# # unique customers
# min/max order date
# Hint:
# Use COUNT(*), COUNT(DISTINCT ...), MIN(...), MAX(...). */

from pyspark.sql import functions as F

# Load Bronze-Layer as Spark DataFrame
df_bronze = spark.table('workspace.ecom_data.raw_orders')

# Check of the Key figures
summary = df_bronze.agg(
    F.count("*").alias("total_rows"),
    F.countDistinct("customer_id").alias("unique_customers"),
    F.countDistinct("order_id").alias("unique_orders"),
    F.min("order_date").alias("min_order_date"),
    F.max("order_date").alias("max_order_date")
)

display(summary)

%python

# Load the raw_orders table with new data from 2025 and verify the same
columns if there are NULL-Values and duplicates

from pyspark.sql import functions as F
from pyspark.sql.window import Window

# Load Bronze table
df_bronze = spark.table('workspace.ecom_data.raw_orders')

```



```

# Remove invalid rows
cleaned = df_bronze.filter(
    (F.col('customer_id').isNotNull()) &
    (F.col('order_id').isNotNull()) &
    (F.col('order_date').isNotNull()) &
    (F.col('sales').isNotNull()) &
    (F.col('sales') > 0)
)

# Deduplicate orders using window function
window = Window.partitionBy('order_id').orderBy(F.col('order_date').asc())
cleaned = cleaned.withColumn('row_num', F.row_number().over(window))
deduped = cleaned.filter(F.col('row_num') == 1).drop('row_num')

# Save the deduped data as a new Delta table in the schema as Silver--Layer
with the table name clean_orders

deduped.write.format('delta').mode('overwrite').saveAsTable('workspace.ecom_data.clean_orders')
deduped.display()

%python
from pyspark.sql import functions as F

# Load Silver table

df_silver = spark.table('workspace.ecom_data.clean_orders')

# Check summary statistics
summary = df_silver.agg(
    F.count("*").alias("rows"),
    F.countDistinct("order_id").alias("unique_orders"),
    F.countDistinct("customer_id").alias("customers"),
    F.min("order_date").alias("min_order_date"),
    F.max("order_date").alias("max_order_date")
)

```

```

display(summary)

%python
# Update the Gold-layer cohort_customers in ecom_data Schema with data from
2025

from pyspark.sql import functions as F
from pyspark.sql.window import Window

df_silver = spark.table('workspace.ecom_data.clean_orders')

# assign a unique, sequential number to each order belonging to a specific
customer, ordered by the date the purchase was made.

w = Window.partitionBy('customer_id').orderBy('order_date')

# The function F.row_number () assigns the number 1 to the earliest order, 2
to the second, and so on.
orders = df_silver.withColumn('row_num', F.row_number().over(w))

# Since I previously ordered the window by order_date, the row with row_num
== 1 is mathematically the earliest purchase for that specific customer and
keep only the column which is necessary for this analysis, the customer_id as
primary key and renamed these first order dates to first_purchase_date

first_orders = orders.filter(F.col('row_num') == 1).select(
    'customer_id',
    F.col('order_date').alias('first_purchase_date')
)

# It does the same with the second_purchase_date, see above and take the
row_num == 2

second_orders = orders.filter(F.col('row_num') == 2).select(
    'customer_id',
    F.col('order_date').alias('second_purchase_date')
)

# Use Left Join for the two tables

cohort = first_orders.join(second_orders, 'customer_id', 'left')

```

```

# take the first purchase date as new column cohort_month and it should have
only the year and month with the date_trunc function

cohort = cohort.withColumn(
    'cohort_month',
    F.date_trunc('month', F.col('first_purchase_date'))
)

# Calculate the difference between the first and second purchase with the
datediff function

cohort = cohort.withColumn(
    'days_to_second_purchase',
    F.datediff('second_purchase_date', 'first_purchase_date')
)

cohort.write.format('delta').mode('overwrite').saveAsTable('workspace.ecom_data.cohort_customers')
cohort.display()

-- Main Validation Query for Gold Layer

WITH validation_stats AS (
    SELECT
        COUNT(*) AS total_customers,
        -- Count customers where the second purchase date is NULL
        COUNT(CASE WHEN second_purchase_date IS NULL THEN 1 END) AS
count_without_second_purchase,
        -- Count customers who HAVE a second purchase date
        COUNT(second_purchase_date) AS count_with_second_purchase,
        -- Check for 2025 data
        MIN(cohort_month) AS earliest_cohort,
        MAX(cohort_month) AS latest_cohort,
        COUNT(CASE WHEN YEAR(cohort_month) = 2025 THEN 1 END) AS rows_in_2025
    FROM workspace.ecom_data.cohort_customers
)

SELECT * FROM validation_stats;

-- Preview a few rows
SELECT * FROM workspace.ecom_data.cohort_customers
LIMIT 10;

```

